

pass, try – except y finally

1 La instrucción pass

¿Qué es?

`pass` es una **instrucción nula**:

cuando Python la ejecuta **no hace nada**, simplemente **sirve para “rellenar”** un bloque de código que, por sintaxis, no puede estar vacío.

¿Para qué sirve?

- Cuando estás **preparando una estructura** (una función, clase, bucle o condicional) pero todavía **no sabes qué código irá dentro**.
- Evita errores como *“IndentationError”* o *“SyntaxError”* por dejar un bloque vacío.

Ejemplo:

```
for i in range(5):  
    pass # Aquí no hacemos nada todavía, pero el bucle es válido
```

Otro ejemplo:

```
def funcion_en_construccion():  
    pass # Ya la implementaré después
```

 Es útil cuando estás **escribiendo código en desarrollo** o haciendo pruebas.

Ejercicio

 **EJERCICIO 12:** Uso de 'pass' para evitar errores de bloque vacío

Objetivo:

Crear la estructura de un programa con condicionales o bucles sin que el intérprete dé error, aunque todavía no haya código dentro.

"""

Crear la estructura de un programa con varias opciones en un menú, sin necesidad de que todas las partes estén todavía implementadas. El programa debe poder ejecutarse sin errores, aunque algunos bloques no contengan instrucciones.

El programa muestra un menú con tres opciones:

- ❶ Calcular algo
- ❷ Hacer otra cosa
- ❸ Salir

■ Para qué sirve:

El `pass` te deja tener un bloque de código “vacío” sin que Python se queje.
Si no lo pones, te daría error de indentación o sintaxis.

❷ `try – except` (manejo de excepciones)

💡 ¿Qué es?

En Python, si ocurre un error durante la ejecución, el programa se detiene y muestra un mensaje de error (una *excepción*). El bloque `try-except` permite **atrapar ese error** y **manejarlo sin que el programa se rompa**.

⚙️ Estructura general:

```
try:
    # código que puede generar un error
except:
    # código que se ejecuta si ocurre un error
```

💻 Ejemplo práctico

```
try:
    numero = int(input("Introduce un número entero: "))
    print(f"El doble es {numero * 2}")
except:
    print("❌ Error: eso no era un número entero.")
```

🏠 Ejemplo de ejecución:

```
Introduce un número entero: hola
❌ Error: eso no era un número entero.
```

👉 El programa **no se detiene**, sino que **maneja el error** de forma controlada.

Situación	Resultado	Explicación	
<code>try</code> antes del <code>input()</code>	✅ Correcto	Protege la conversión de texto a número	
<code>try</code> después del <code>input()</code>	❌ Error no capturado	El fallo ocurre fuera del bloque <code>try</code>	

Si lo hiciéramos con **if**

Para hacer lo mismo sin `try/except`, tendríamos que **comprobar manualmente si la entrada es numérica**, antes de convertirla:

```
texto = input("Introduce un número: ")

if texto.isdigit():
    numero = int(texto)
    print("El doble es:", numero * 2)
else:
    print("❌ Error: Debes escribir un número entero.")
```

📦 Ejemplo:

```
Introduce un número: hola
❌ Error: Debes escribir un número entero.
```

💡 También puedes manejar errores específicos

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("❌ No se puede dividir entre cero.")
```

🧠 Principales excepciones integradas en Python

🌱 Excepción	💬 Cuándo ocurre	💻 Ejemplo que la provoca
<code>ZeroDivisionError</code>	Cuando se intenta dividir entre 0	<code>10 / 0</code>
<code>ValueError</code>	Cuando se intenta convertir un valor de tipo incorrecto	<code>int("hola")</code>
<code>TypeError</code>	Cuando se mezclan tipos incompatibles	<code>"texto" + 5</code>
<code>NameError</code>	Cuando se usa una variable que no existe	<code>print(variable_inexistente)</code>
<code>IndexError</code>	Cuando se accede a una posición fuera del rango de una lista	<code>[1, 2, 3][5]</code>
<code>KeyError</code>	Cuando se accede a una clave inexistente en un diccionario	<code>{"a": 1}["b"]</code>
<code>FileNotFoundError</code>	Cuando se intenta abrir un archivo que no existe	<code>open("archivo_que_no_existe.txt")</code>
<code>AttributeError</code>	Cuando se usa un método o atributo que no pertenece a un objeto	<code>"hola".append(3)</code>
<code>ImportError</code>	Cuando no se puede importar un módulo	<code>import modulo_que_no_existe</code>
<code>ModuleNotFoundError</code>	Subtipo de <code>ImportError</code> : el módulo no se encuentra	<code>import inexistente</code>
<code>SyntaxError</code>	Cuando el código tiene errores de sintaxis	<code>if 5 > 3 print("error")</code>

🧠 Puedes añadir **más de un tipo de excepción**, o incluso usar `except Exception as e`: para ver el mensaje exacto del error.

```
except Exception as e:
```

→ Le estás diciendo a Python:

“Si ocurre **cualquier error** (de cualquier tipo),
guárdame la información del error en la variable `e`.”

Y luego puedes **mostrarla, imprimirla o analizarla**.

Ejemplo práctico

python

```
try:
    numero = int(input("Introduce un número: "))
    resultado = 10 / numero
    print("El resultado es:", resultado)
except Exception as e:
    print("⚠ Ha ocurrido un error:", e)
```

Ejercicio 13 con try y except — “Evitar que el programa se rompa”

"""

 EJERCICIO 2: Manejar errores con 'try' y 'except'

Escribe un programa que pida al usuario que introduzca un número entero e intenta dividir 10 entre ese número. Si el usuario introduce texto o algo que no sea un número, se genera una excepción 'ValueError'.

Si el usuario introduce el número 0, se produce una excepción 'ZeroDivisionError'.

En ambos casos, el programa muestra un mensaje de error personalizado y continúa funcionando

sin interrumpirse.

Ejemplo de funcionamiento:

Introduce un número entero: 5

El resultado de dividir 10 entre 5 es: 2.0

Introduce un número entero: 0

✗ No se puede dividir entre 0.

Introduce un número entero: hola

✗ Debes introducir un número entero válido.

Añade otro programa similar al final usando `except Exception as e` y que sea el program el que te diga que error ha ocurrido.

Observa:

🧠 Errores comunes en Python y sus mensajes

✳ Tipo de error (nombre técnico)	💡 Cuándo ocurre	💻 Ejemplo de código que lo provoca	📄 Mensaje que muestra Python (e)
<code>ZeroDivisionError</code>	Cuando intentas dividir un número entre 0.	<code>10 / 0</code>	<code>division by zero</code>
<code>ValueError</code>	Cuando intentas convertir un texto a número, y ese texto no representa un valor numérico válido.	<code>int('j')</code>	<code>invalid literal for int() with base 10: 'j'</code>

3 finally

💡 ¿Qué es?

El bloque **finally** siempre se ejecuta, ocurra o no un error.
Se usa para **cerrar recursos, limpiar datos o mostrar mensajes finales**.

⚙️ Estructura completa:

```
try:
    # código que puede fallar
except:
    # código si ocurre error
finally:
    # código que siempre se ejecuta
```

💻 Ejemplo:

```
try:
    numero = int(input("Introduce un número: "))
    resultado = 10 / numero
    print("El resultado es:", resultado)
except ZeroDivisionError:
    print("❌ No puedes dividir entre 0.")
except ValueError:
    print("❌ Debes introducir un número.")
finally:
    print("✅ Fin del programa. Gracias por usarlo.")
```

📦 Ejemplo de ejecución 1:

```
Introduce un número: 0
❌ No puedes dividir entre 0.
✅ Fin del programa. Gracias por usarlo.
```

📦 Ejemplo de ejecución 2:

```
Introduce un número: 5
El resultado es: 2.0
✅ Fin del programa. Gracias por usarlo.
```

💡 El bloque **finally** siempre se ejecuta, incluso si hay un **return** o un **break**.

Resumen:

-  `pass` → “de momento no hago nada”
-  `try/except` → “intento hacer algo, y si falla, lo manejo”
-  `finally` → “pase lo que pase, esto se ejecuta siempre”

Ejercicio 14 con `finally` — “Bloque que siempre se ejecuta”

"""

 EJERCICIO 3: Uso de 'finally' para acciones que siempre deben ejecutarse tanto si ocurre un error dentro del bloque 'try' como si no. **Para qué sirve:** `finally` se usa cuando algo debe **ocurrir sí o sí**, como cerrar un archivo, liberar memoria o mostrar un mensaje de salida.

Escribir un programa que muestre un mensaje inicial, pedir un número, y calcular su doble. Si el usuario introduce un valor no numérico, se muestra un mensaje de error, pero el bloque 'finally' se ejecuta igualmente.

Para qué sirve:

`finally` se usa cuando algo debe **ocurrir sí o sí**, como cerrar un archivo, liberar memoria o mostrar un mensaje de salida.